Project – Low Level Design

On

CI/CD for Python Retail App

Course Name : DevOps

Institution Name: Medicaps University–Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
01	Mohit Meharde	EN22CS301609
02	Nancy Agrawal	EN22CS301634
03	Niharika Panwar	EN22CS301645
04	Nikhil Garg	EN22CS301646
05	Mahima Sahu	EN22CS301570
06	Naman Mangroliya	EN22CS301628

Group Name: Group 10D6

Project Number: DO-21

Industry Mentor Name: Mr. Vaibhav

University Mentor Name : Akshay Saxena

Academic Year: 2025-2026

1. Introduction

1.1 Scope of the Document

This document defines the high-level architecture and design for automating infrastructure provisioning and deployment of a Python-based Retail Application. It focuses on implementing Continuous Integration and Continuous Deployment (CI/CD) using Infrastructure as Code (IaC) to ensure a scalable, reliable, and repeatable deployment process.

Additionally, the document outlines the integration of DevOps tools and practices to streamline development workflows, reduce manual intervention, and improve software delivery speed. It also covers deployment strategies, monitoring, and security considerations required for modern retail systems.

1.2 Intended Audience

- **Project Supervisors:** To evaluate system feasibility, architecture design, and implementation strategy
- **DevOps Engineers:** To understand CI/CD workflows, automation pipelines, and infrastructure provisioning
- **System Architects:** To review integration of Docker, Kubernetes, Terraform, and pipeline tools
- **Developers:** To understand deployment processes and integration requirements

1.3 System Overview

The system automates the build, test, and deployment of a Retail Application developed using Python. It leverages CI/CD pipelines to integrate code changes, build Docker images, and deploy them onto Kubernetes clusters using Terraform.

The system ensures:

- Faster release cycles
- Continuous testing and validation
- Consistent deployment across environments
- Reduced human errors

It supports multiple environments such as **Development, Testing, and Production**, ensuring smooth transitions between stages.

2. System Design

The system design of the **CI/CD Pipeline for Python Retail Application integrated with an Autonomous Retail Researcher Agent** is focused on building a robust, scalable, and automated solution for retail data analysis and deployment. The design combines artificial intelligence-based data processing with modern DevOps practices such as continuous integration, continuous deployment, and containerization.

The architecture ensures that the system can efficiently handle large volumes of retail data, process it into meaningful insights, and deliver those insights through an interactive dashboard. At the same time, the DevOps pipeline guarantees that any changes in the application are automatically tested, built, and deployed without manual intervention.

The system follows a **layered and modular architecture**, where each component performs a specific role, ensuring better maintainability, scalability, and fault isolation.

2.1 Application Design

The application is designed using a **modular layered architecture**, which separates different functionalities into independent layers. This improves flexibility and allows individual components to be updated without affecting the entire system.

1. Presentation Layer

This layer is responsible for user interaction and visualization.

- Built using **Streamlit dashboard**
- Displays:
  - KPIs (Key Performance Indicators)
  - Product comparisons
  - Price trends and graphs
- Provides an intuitive interface for users to interact with insights

2. Application (Business Logic) Layer

This layer handles the core logic of the system.

- Processes incoming data
- Applies filtering and transformation rules
- Integrates AI-based analysis for generating insights
- Acts as a bridge between data layer and presentation layer

3. Data Layer

This layer manages all data-related operations.

- Stores raw and processed data
- Uses formats like:

  - CSV files
  - Databases (optional for scalability)
- Ensures efficient data retrieval for analysis

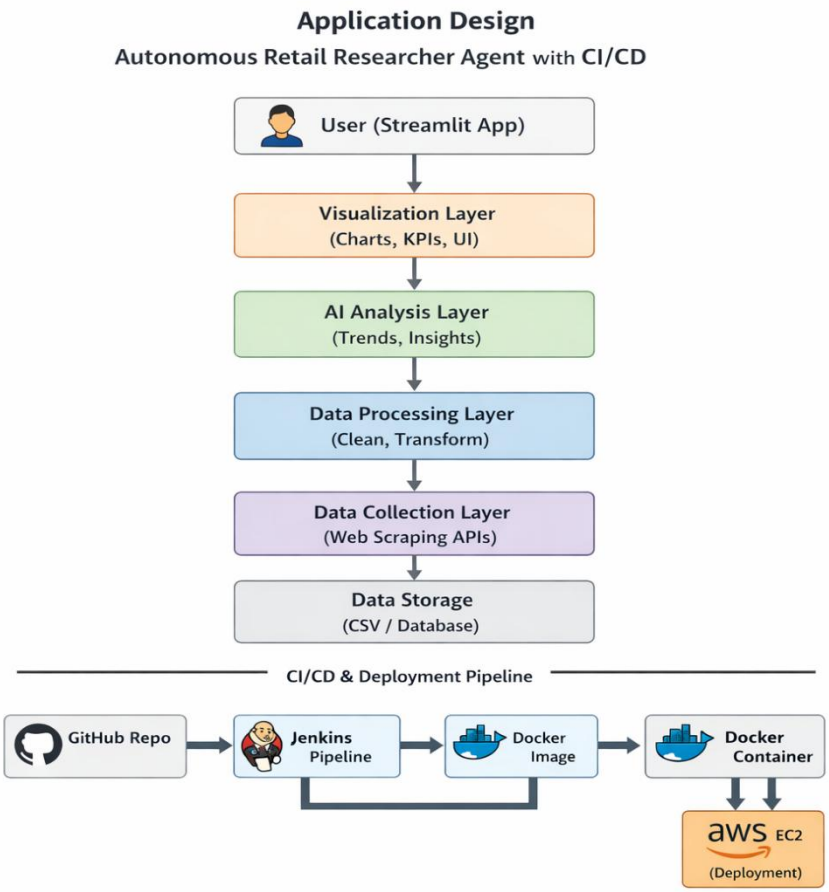
4. DevOps Layer

This layer automates software delivery.

- Uses **Jenkins CI/CD pipeline**
- Integrates:
  - GitHub for version control
  - Docker for containerization
  - AWS EC2 for deployment

Design Principles

- Loose coupling between modules
- High cohesion within components
- Reusability of services
- Easy scalability and maintenance



2.2 Process Flow

The process flow explains the complete lifecycle of the system from development to deployment and execution.

Development to Deployment Flow

1. Developer writes or modifies application code
2. Code is pushed to GitHub repository
3. GitHub webhook triggers Jenkins pipeline
4. Jenkins performs:
    - Code checkout
    - Dependency installation
    - Automated testing
5. Docker image is built using Dockerfile
6. Docker container is created from image
7. Container is deployed on AWS EC2 server
8. Application starts running on specified port
9. Users access the application through browser

Execution Flow (Application Side)

- User accesses dashboard
- System fetches processed data
- AI module generates insights
- Dashboard displays results

Advantages

- Fully automated deployment
- Faster release cycles
- Reduced human errors
- Continuous monitoring and improvement

2.3 Information Flow

The information flow in the **CI/CD Pipeline integrated Autonomous Retail Researcher Agent system** defines how data and control signals move across different modules, starting from data acquisition to final visualization and deployment. A well-structured information flow ensures that the system operates efficiently, maintains data consistency, and provides accurate insights to end users.

The system follows a **pipeline-based flow**, where each stage processes the output of the previous stage. This structured approach enables smooth communication between modules and supports automation through the DevOps pipeline.

2.3.1 Data Flow within the Application

1. Data Collection Stage

- The process begins with collecting retail data from external sources such as e-commerce websites.
- Web scraping tools (e.g., BeautifulSoup) extract:
  - Product name
  - Price
  - Rating
  - Reviews
- The collected data is initially unstructured and may contain noise.

2. Data Processing Stage

- Raw data is passed to the processing module for cleaning and transformation.
- Key operations include:
  - Removing duplicate entries
  - Handling missing or null values
  - Formatting data into structured form
- The processed data becomes ready for further analysis.

3. Data Storage Stage

- Cleaned data is stored in structured formats such as:
  - CSV files
  - Databases (if extended)
- Storage ensures:
  - Data persistence
  - Easy retrieval for analysis and visualization

4. Data Analysis Stage

- The processed data is forwarded to the AI analysis module.
- The module generates insights such as:
  - Price comparison across products
  - Market trends

  - Best-performing products
- This stage transforms raw data into meaningful information.

5. Data Visualization Stage

- The analyzed data is sent to the dashboard module.
- Visualization tools convert data into:
  - Graphs (bar, line, pie charts)
  - KPIs (average price, top-rated product)
- The user interacts with the dashboard to explore insights.

2.3.2 DevOps Information Flow

In addition to data flow, the system also includes a **control flow for deployment and updates** using CI/CD.

Steps:

1. Developer updates code and pushes to GitHub
2. GitHub webhook triggers Jenkins pipeline
3. Jenkins retrieves code and initiates build process
4. Dependencies are installed and tests are executed
5. Docker image is created
6. Image is deployed as a container
7. Container runs on AWS EC2 instance
8. Updated application becomes available to users

This flow ensures that any changes in the application are automatically reflected in the deployed system.

2.3.3 Integrated Flow Overview

The system integrates both **data flow and DevOps flow**, ensuring continuous operation.

- Data Flow → Handles retail data processing and insights generation
- DevOps Flow → Handles application updates and deployment
- Both flows work together to provide:
  - Real-time insights
  - Continuous delivery of updates

2.3.4 Key Characteristics of Information Flow

- **Automation:** Entire pipeline is automated using CI/CD tools
- **Continuity:** Data flows continuously without manual intervention
- **Scalability:** Can handle increasing data volume and users
- **Efficiency:** Optimized processing ensures faster results
- **Reliability:** Ensures consistent and accurate data delivery

2.4 Components Design

The system is designed using a modular approach, where each component performs a specific function within the overall architecture of the **Autonomous Retail Researcher Agent integrated with a CI/CD pipeline**. This modular design improves scalability, maintainability, and flexibility, allowing independent development and deployment of system components. Each component interacts with others through well-defined interfaces, ensuring smooth data flow and system efficiency.

2.4.1 Data Acquisition Component

This component is responsible for collecting retail data from external sources such as e-commerce websites. It uses web scraping techniques to extract product-related information.

- Utilizes tools like **BeautifulSoup or Scrapy**
- Extracts:
    - Product name
    - Price
    - Ratings and reviews
- Handles both static and dynamic web content

This component ensures that relevant and up-to-date data is available for further processing.

2.4.2 Data Processing Component

The data processing component cleans and transforms raw data into a structured format suitable for analysis.

- Performs:
    - Data cleaning (removing duplicates, handling missing values)
    - Data formatting and normalization
- Converts unstructured data into structured datasets

2.4.3 AI Analysis Component

This is the core intelligence module of the system, responsible for generating insights from processed data.

- Performs:
  - Price comparison
  - Trend analysis
  - Product performance evaluation
- Can be extended with machine learning models

The output of this component provides meaningful insights that support decision-making.

2.4.4 Data Storage Component

This component manages the storage of both raw and processed data.

- Stores data in:
  - CSV files
  - Databases (optional for scalability)
- Ensures:
  - Data persistence
  - Efficient retrieval

It acts as a central repository for all system data.

2.4.5 Visualization Component

The visualization component provides a user-friendly interface for interacting with the system.

- Built using **Streamlit**
- Uses visualization libraries such as:
  - Plotly
  - Altair
- Displays:
  - Charts and graphs
  - KPIs and summaries

This component enhances user experience by presenting complex data in an understandable format.

2.4.6 CI/CD Pipeline Component

This component automates the build, test, and deployment processes.

- Implemented using **Jenkins**
- Performs:
  - Code integration
  - Automated testing
  - Deployment automation
- Triggered by GitHub webhooks

It ensures continuous integration and delivery of the application.

2.4.7 Containerization Component

The containerization component ensures that the application runs consistently across different environments.

- Uses **Docker**
- Includes:
  - Dockerfile for environment setup
  - Image creation
  - Container execution
- Eliminates dependency issues

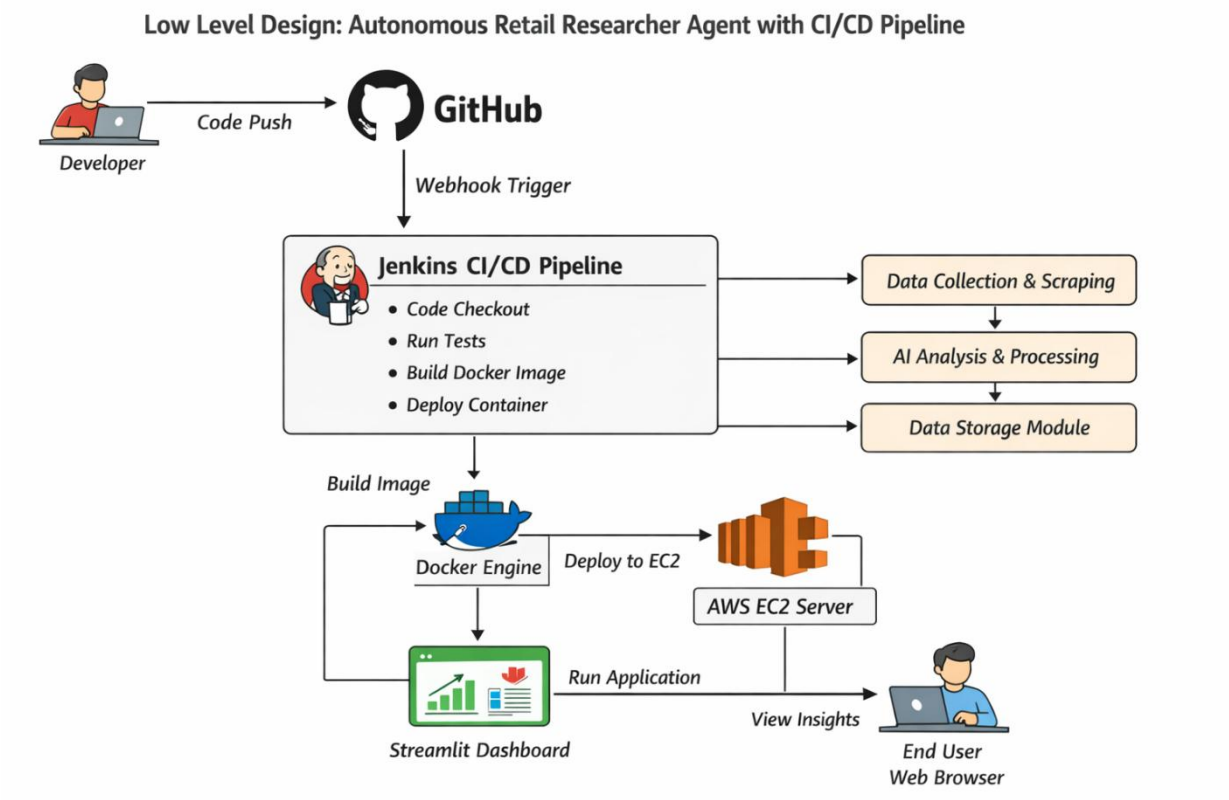
This component improves portability and deployment reliability.

2.4.8 Deployment Component

This component handles the hosting and execution of the application in a cloud environment.

- Uses **AWS EC2** for deployment
- Runs Docker containers on cloud instances
- Provides:
  - Scalability
  - High availability

It ensures that the application is accessible to users over the internet.



2.5 Key Design Considerations

The system is designed with several important considerations:

1. Scalability

- Cloud deployment using AWS
- Can handle increasing users and data load

2. Reliability

- Automated CI/CD reduces deployment failures
- System ensures continuous availability

3. Performance

- Optimized data processing
- Fast dashboard response

4. Security

- Secure AWS instance access
- Controlled Jenkins pipeline execution
- API-level security (can be enhanced)

5. Maintainability

- Modular architecture allows easy updates
- Independent components reduce complexity

6. Portability

- Docker containers run on any environment
- Eliminates system dependency issues

7. Extensibility

- New features like ML models or APIs can be added easily

2.6 API Catalogue

The system exposes multiple APIs for internal communication and integration.

Internal APIs

1. /scrape-data

- Collects data from external sources
- Input: URL or category
- Output: Raw data

2. /process-data

- Cleans and structures data
- Output: Processed dataset

3. /analyze-data

- Generates insights using analysis logic
- Output: Analytical results

4. /get-dashboard-data

- Sends processed data to frontend
- Output: JSON/structured data

DevOps Integration APIs

1. **GitHub Webhook**
    - Triggers Jenkins pipeline
2. **Jenkins API**
    - Controls build and deployment
3. **Docker API**
    - Manages image and container lifecycle

Output APIs

1. /get-kpi

- Returns summary metrics

2. /get-visualization

- Provides data for charts

References

[1] Amazon Web Services, *"AWS Documentation,"* Available: AWS Official Documentation

[2] Jenkins, *"Jenkins User Documentation,"* Available: Jenkins Documentation

[3] Docker Inc., *"Docker Documentation,"* Available: Docker Docs

[4] GitHub, *"GitHub Documentation and Guides,"* Available: GitHub Docs

[5] TutorialsPoint, *"DevOps Tools Overview,"* Available: DevOps Tools Guide

[6] J. Smart, *"Jenkins: The Definitive Guide,"* O'Reilly Media, 2011.

[7] K. Matthias and S. Kane, *"Docker: Up and Running,"* O'Reilly Media, 2018.

[8] S. Sharma and B. Coyne, *"DevOps for Dummies,"* Wiley Publishing, 2015.

[9] M. Hüttermann, *"DevOps for Developers,"* Apress, 2012.

[10] Git SCM, *"Git Tutorial Documentation,"* Available: Git Official Tutorial